MalVision: Using Deep Convolutional Neural Networks and Image Visualization

March 4, 2026

Team Members

Agnij T Dev	[TVE23CS014]
Anzer Farooq Ganie	[TVE23CS041]
Maleeha Beefathima	[TVE23CS088]
Niya P Sherry	[TVE23CS107]

Project Guide

Prof. Bincy P Mathew
Assistant Professor
Department of Computer Science and Engineering
College of Engineering Trivandrum

Contents

Introduction to the Problem Area

Application Domain : Cybersecurity & Static Malware Analysis

Need for a Digital/Automated Solution: The sheer volume of new, sophisticated malware (like zero-day exploits and polymorphic viruses) makes automated, rapid detection critical to preventing system breaches.

Difficulties of Traditional Approaches: Traditional signature-based detection is easily bypassed by code obfuscation. Manual dynamic analysis is time-consuming, requires isolated environments, and cannot scale to meet modern threat levels.

Motivation & Literature Survey

Existing Solutions Features

- Signature-Based Detection: Scans files against databases of known malicious byte sequences or hashes.
- Dynamic Analysis (Sandboxing): Executes suspicious files in isolated virtual environments to monitor their behavior.
- Traditional Machine Learning: Uses manually extracted code features (like API calls or n-grams) to train predictive models.

Drawbacks of These Approaches

- Signature-Based: Completely blind to new (zero-day) attacks and disguised (polymorphic) malware.
- Dynamic Analysis: Extremely slow, resource-heavy, and easily evaded by malware that detects the sandbox.
- Traditional ML: Relying on manual feature extraction creates a complex, time-consuming bottleneck.

Motivation & Literature Survey

Paper & Contribution	Key Methodology	Merits (Advantages)	Demerits (Disadvantages)
Malware Images (2011) *Pioneered Mapping*	Maps raw byte values (0–255) to grayscale pixel intensities for texture analysis.	**Highly Efficient:** 40x faster than n-gram methods; 98% accuracy.	**Vulnerable:** Susceptible to section relocation or padding.
PE Classification (2022) *Deep Learning*	Converts PE binaries into fixed-size grayscale images for CNN input.	**Safe Analysis:** 95.07% accuracy via static analysis. Robust.	**High Overhead:** Large image sizes increase training time.
CNN-BiLSTM (2024) *Hybrid Framework*	Combines structural features (CNN) with sequential opcodes (BiLSTM).	**Superior Accuracy:** 98.7% by fusing structure and behavior.	**High Complexity:** Requires both static and runtime data.
Vision Transformer (2022) *ViT vs. CNN*	Compares VGG19, custom CNNs, and ViTs on Malimg dataset.	**Generalization:** VGG19 achieved 99% via Transfer Learning.	**ViT Failure:** Failed to converge due to data scarcity.
Image Ensemble (2020) *Fine-Tuned Models*	Ensemble of fine-tuned VGG16 and ResNet-50 models.	**Performance:** >99% for unpacked; Fast (1.18s) inference.	**Cost:** Expensive training; struggles with subtle differences.

Problem Statement

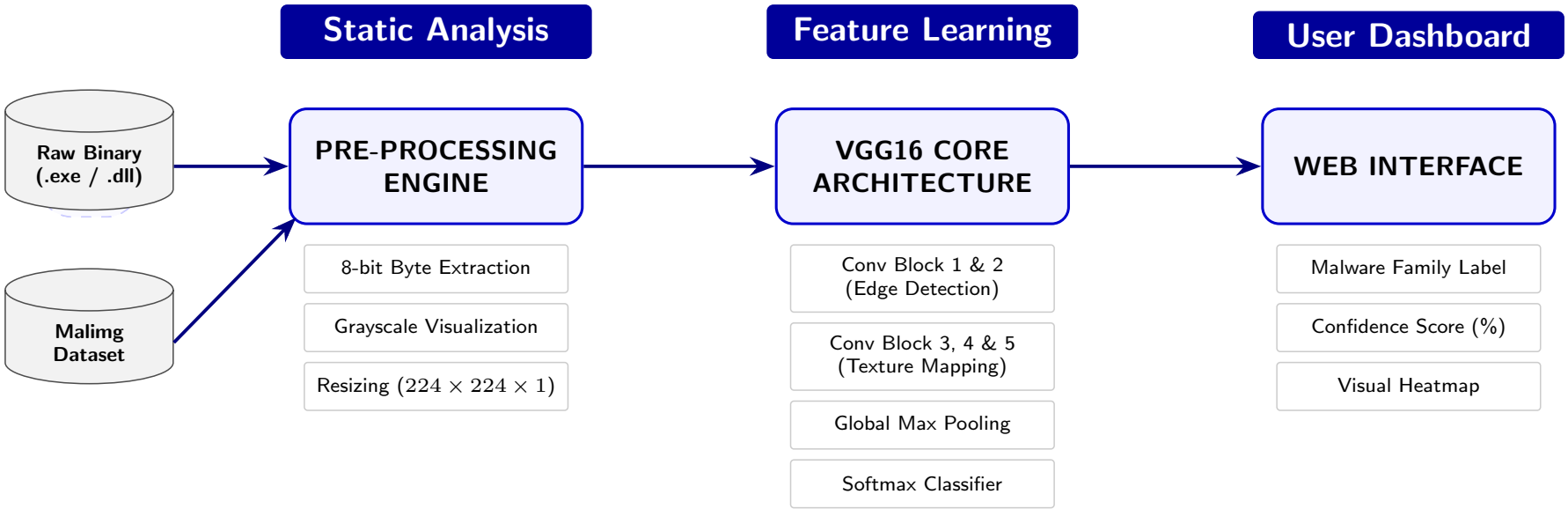
The problem is stated as to design and develop a machine learning-based application that classifies malware by converting binaries into images, utilizing deep learning (such as CNNs) to identify malicious patterns without executing the code or relying on traditional signatures.

Functionalities

The system will provide the following core functionalities:

1. **Upload**: Users import raw binaries like .exe or .dll files into the system for static analysis.
2. **Convert**: The preprocessing layer transforms raw bytes into 8-bit grayscale image representations.
3. **Standardize**: Images are automatically resized to 224x224 pixels to ensure deep learning compatibility.
4. **Analyze**: The VGG16 engine extracts structural "textures" to identify unique malware family patterns.
5. **Display**: Outputs the final malware family label along with a statistical confidence score indicating the model's certainty.

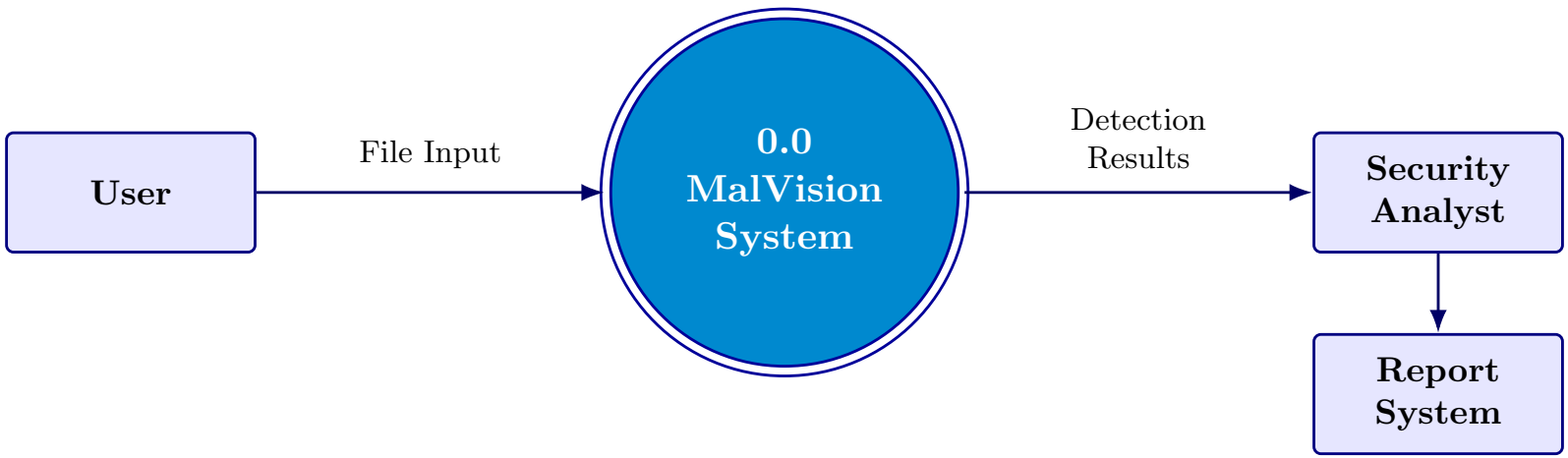
Detailed System Architecture



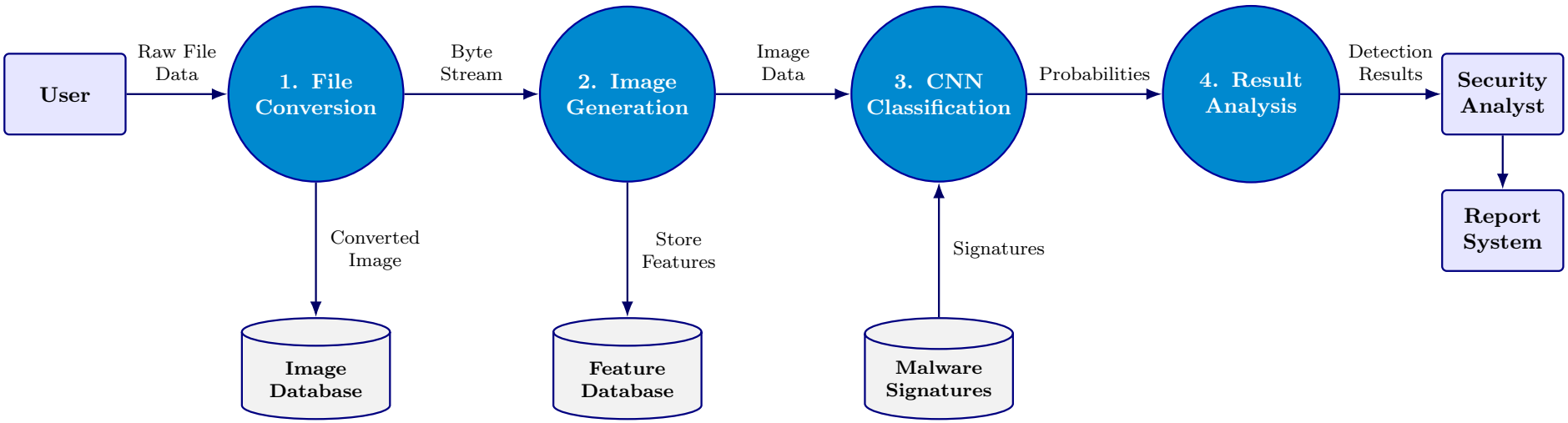
Module Description

Module	Input	Processing	Output
File Conversion	Raw File Data	**Data Parsing:** Parses binary data. **Image Transformation:** Converts stream to 2D format. **Storage:** Saves generated images.	Grayscale Image
Image Generation	Converted Image	**Normalization:** Standardizes images. **Feature Extraction:** Extracts textures/patterns.	Processed Image / Features
CNN Classification	Feature Vectors	**CNN Model:** Processes features. **Prediction:** Matches malware signatures.	Detection Results
Result Analysis	Results	**Classification:** Refines predictions. **Report:** Generates final output.	Final Reports

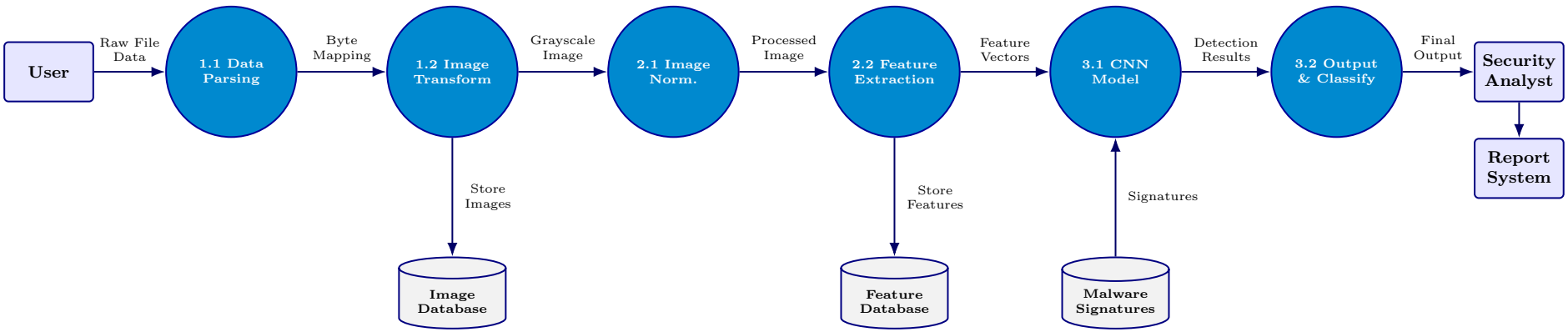
Data Flow Diagrams: Level 0 (Context Diagram)



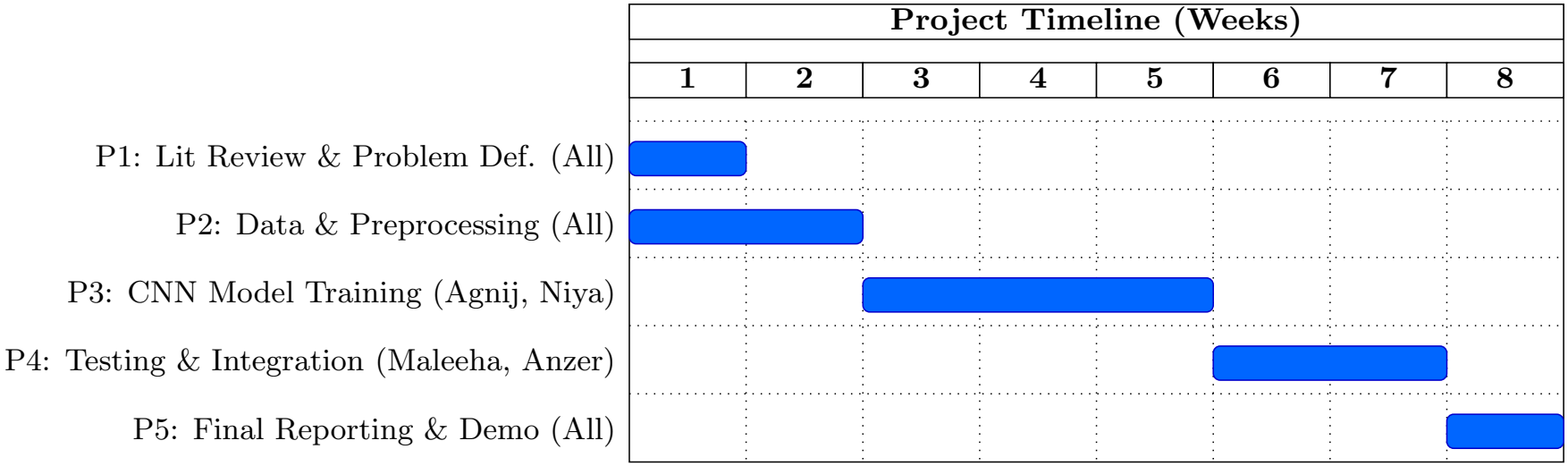
Data Flow Diagrams: Level 1 (High-Level Modules)



Data Flow Diagrams: Level 2 (Detailed Sub-Processes)



Work Plan: Timeline & Module Allocation



Division of Module-Based Work

- **Anzer & Maleeha:** Responsible for Backend integration, Preprocessing Scripts (Binary-to-Image), and UI Frontend design.
- **Agnij & Niya:** Responsible for Data Acquisition, VGG16 fine-tuning, Model evaluation, and parameter optimization.

Project Execution & Work Plan Adherence

Project Status: Fully Completed

Milestones Achieved (Weeks 1–8):

- **Data & Preprocessing (Weeks 1-2):** Successfully parsed the Malimg dataset and deployed the raw byte-to-grayscale conversion scripts exactly on schedule.
- **Deep Learning Core (Weeks 3-5):** Model training completed seamlessly. By leveraging Google Colab's T4 GPU, we bypassed local hardware bottlenecks and achieved our highly accurate (97.28%) model within the allocated timeframe.
- **Testing & Integration (Weeks 6-7):** The Flask API seamlessly linked the UI and backend. We successfully completed rigorous edge-case testing, generated the confusion matrix, and evaluated false positives.
- **Reporting (Week 8):** Final documentation compiled and the system successfully deployed for real-time classification on the localhost server.

Justification of Success:

- Strict adherence to the proposed Gantt chart was maintained by parallelizing tasks. While the core AI team trained the time-intensive CNN in the cloud, the frontend team built the UI. This allowed ample time for our final phase of rigorous testing and deployment.

Software/Hardware Framework

Software Framework

- **Frontend (Lightweight & Zero-Dependency):** HTML5 & Bootstrap 5.3 for a responsive, dark-mode UI. Vanilla JS (ES6+) uses the Fetch API and DOM manipulation for seamless, no-reload async communication with the backend.
- **Backend (API & File Handling):** Python (Flask) to securely route user uploads, parse raw binaries, and serve the deep learning model.
- **Machine Learning:** TensorFlow/Keras for implementing, fine-tuning, and executing the VGG16 CNN model.

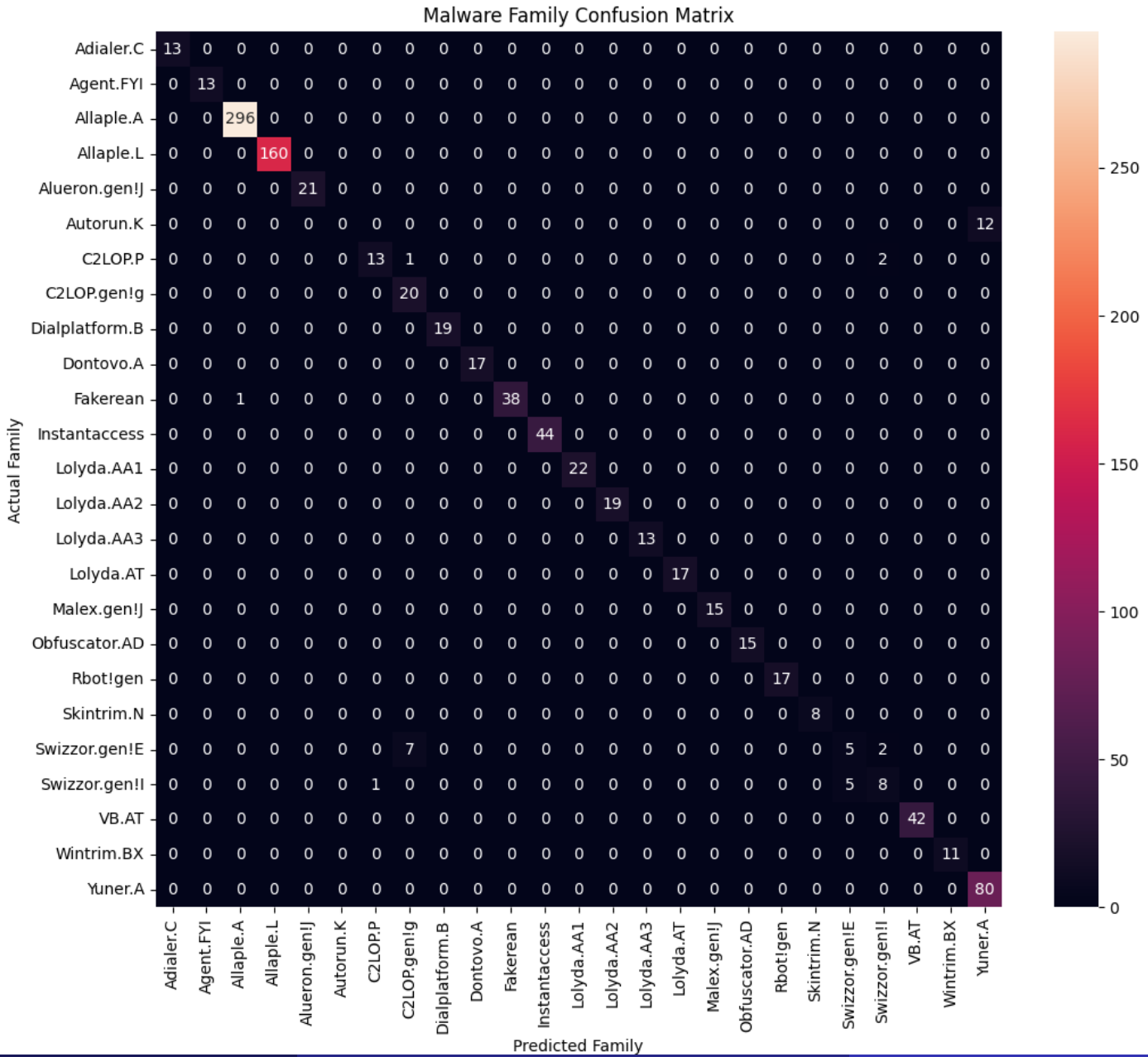
Hardware Framework

- **Model Training:** Google Colab (NVIDIA T4 GPU, 12GB RAM) for cloud-accelerated computing to bypass local hardware limits.
- **System Deployment:** Standard consumer PC (8GB RAM) for running the local localhost inference server and rendering the UI.

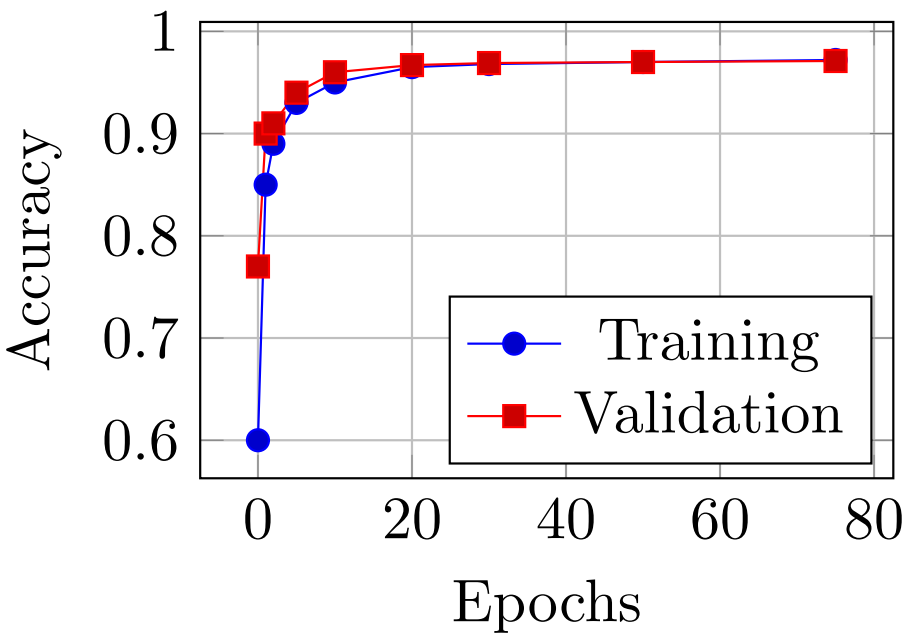
Result

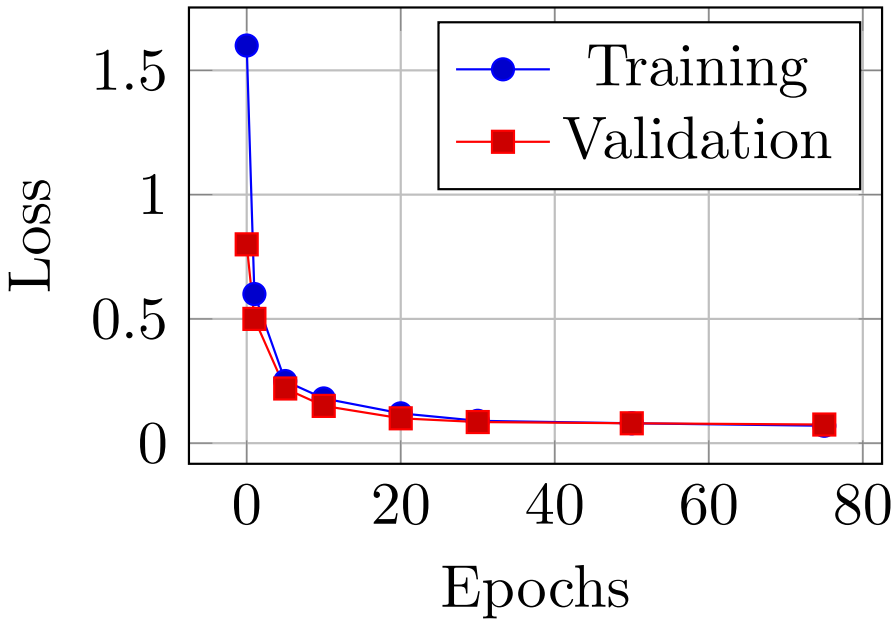
- **System Status**: All modules are successfully integrated and the full application is operational on a local server (localhost).
- **Image Generation**: The system accurately parses uploaded binary executables and renders them into clear 2D grayscale images.
- **Real-Time Classification**: The deep learning model successfully analyzes the generated images and dynamically outputs Malware Family and Confidence Score.
- **UI Integration**: The frontend successfully communicates with the backend, displaying the generated image and analysis results seamlessly without requiring a page reload.

Confusion Matrix



Pattern Accuracy & Loss



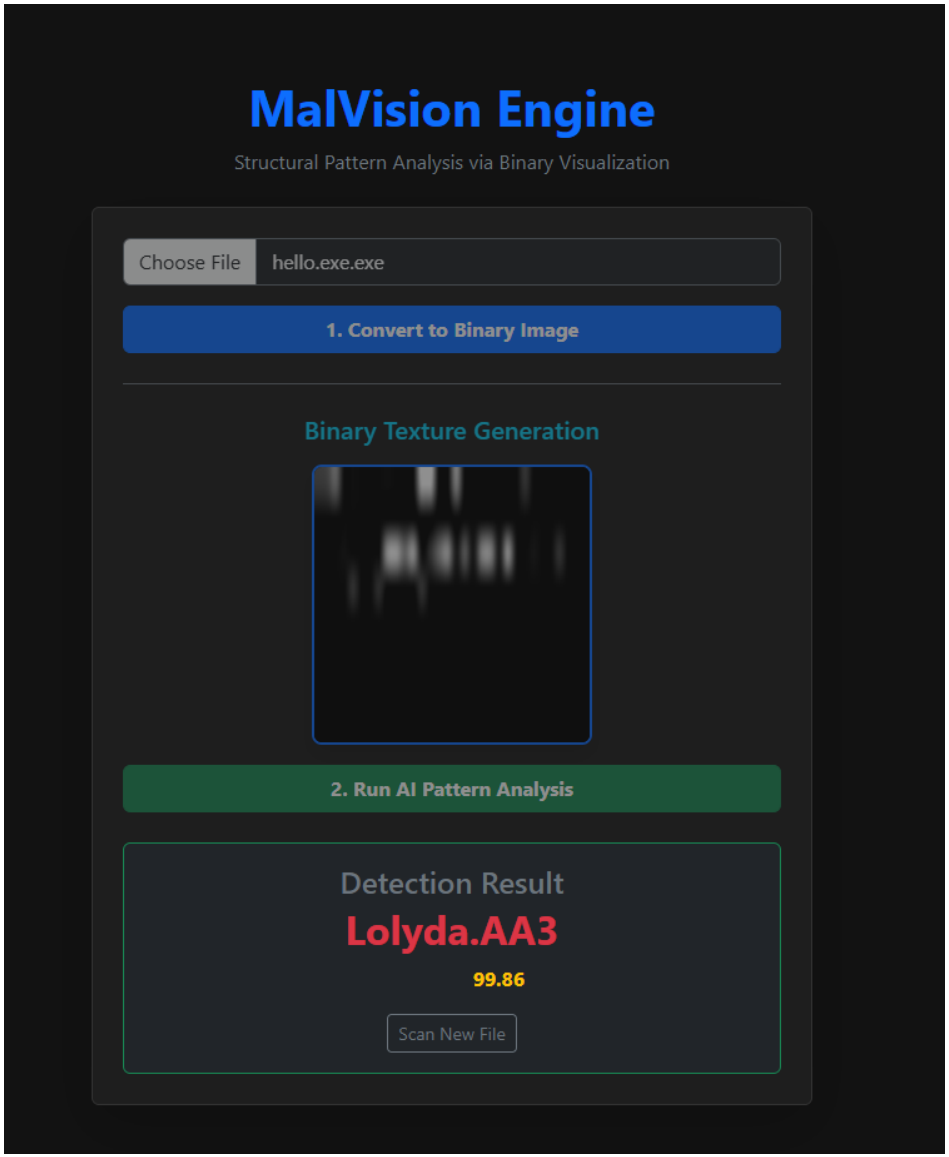


accuracy: 0.9728 - loss: 0.0712

Final Test Accuracy (Pattern-Based): 97.28%

The model generalizes well (Gap: 0.11%).

Output


MalVision Engine
Structural Pattern Analysis via Binary Visualization
Choose File
hello.exe.exe
1. Convert to Binary Image
Binary Texture Generation
2. Run AI Pattern Analysis
Detection Result
Lolyda.AA3
99.86
Scan New File

Conclusion

Present Status The system successfully converts binaries to images and classifies them with 97.28% accuracy, integrating the frontend with the ML backend.

Future Challenges Addressing adversarial attacks (where malware deliberately alters its binary image footprint) and optimizing the model to run efficiently on low-resource devices.

References

- Bensaoud, A., Kalita, J. (2024). *A Survey of Malware Detection Using Deep Learning.* IEEE.
- Chen, Cheng, Liu, Tianxu, Shen, Kaihui, Zhang, Lixia (2024). *Approach to Malicious Code Detection using CNN-BiLSTM and Feature Fusion.* IEEE.
- Davis, R., Roy, K., Xu, J. (2024). *Classifying Malware Traffic Using Images and Deep Convolutional Neural Network.* IEEE.
- Alhudhaif, A., Alzahrani, S., Khan, M., Ullah, F. (2024). *Enhanced Image-Based Malware Classification Using Transformer-Based Convolutional Neural Networks.* Electronics MDPI.
- Almiani, M., Al-Rawashdeh, M., Al-Sbou, Y., Jararweh, Y. (2024). *Lightweight Malware Detection and Family Classification System for IoT.* Journal of Computer and Communications.
- Alqahtani, F., Bensaoud, A., Bouhorma, M., Kalita, J. (2024). *Case Study: Neural Network Malware Detection Verification for Feature and Image Datasets.* IEEE/ACM Formal Methods in

References

- Abdul Alim, Md., Arif, Md Habibul, Hossen, Md Shakhawat, Palomino, B., Rahman, Mamunur, Rahman, Md Reduanur, Stowbunenko, V., Tohfa, Nasrin Akter, Zareen, S. (2023). *Malware Classification by Vision Transformer and Transfer Learning.*
- Chandranegara, D. R., Nugroho, A., Rahman, A. (2023). *Malware Image Classification Using Deep Learning InceptionResNet-V2 and VGG-16 Method.* IEEE.
- Stowbunenko, V. (2023). *Image-Based Classification of Malware using t-SNE Images.* Springer.
- Palomino, B. (2023). *Image-Based Malware Detection using Convolutional Neural Network Techniques.* Master's Thesis.
- Hidayat, R., Nugraha, A., Prasetyo, Y. (2023). *Intelligent Malware Classification Based on Network Traffic and Data Augmentation Techniques.* Indonesian Journal of Electrical Engineering.
- Heydari, V., Ho, S.-S., Kiger, J. (2022). *Malware Binary Image Classification Using Convolutional Neural Networks.* ICCWS.

References

- Abdul Alim, Md., Hossen, Md Shakhawat, Rahman, Md Reduanur, Zareen, S. (2022). *Malware Classification by Vision Transformer and Transfer Learning.* IEEE.
- Bista, R., Pant, D. (2021). *Image-based Malware Classification using Deep Convolutional Neural Network and Transfer Learning.* IEEE.
- Ahmed, S., Hossain, M., Rahman, T. (2021). *Classification and Analysis of Android Malware Images Using Feature Fusion Technique.* IEEE.
- Alazab, M., Safaei, B., Vasan, D., Wassan, S., et al. (2020). *Image-Based Malware Classification Using Ensemble of CNN Architectures (IMCEC).*
- Bouhorma, M., Prima, B. (2020). *Using Transfer Learning for Malware Classification.* IEEE.
- Jain, M. (2019). *Image-Based Malware Classification with Convolutional Neural Networks and Extreme Learning Machines.* San Jose State University.

References

- Bruce, N., Kalash, M., Mohammed, N., Rochan, M., Wang, Y. (2018). *Malware Classification Framework using Convolutional Neural Network.* IEEE.
- Amin, M., Azmoodeh, A., Dehghantanha, A. (2017). *Binary Malware Image Classification using Machine Learning with Local Binary Pattern.* IEEE Big Data Conference.
- Jacob, G., Karthikeyan, S., Manjunath, B. S., Nataraj, L. (2011). *Malware Images: Visualization and Automatic Classification.*
- Vision Research Lab, UCSB. *Malimg Dataset.*

Thank You